PROJECT REPORT

On

Root ED

Submitted by

Milee Bhatt (IU2241230225)
Dev Machchhar (IU2241230181)
Gargie (IU2241230183)

In fulfillment for the requirements of Software Group Project as a part

Of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE & ENGINEERING

ज्ञानेन प्रकाशते जगत्

INDUS UNIVERSITY

INSTITUTE OF TECHNOLOGY AND ENGINEERING
INDUS UNIVERSITY CAMPUS, RANCHARDA, VIA-THALTEJ
AHMEDABAD-382115, GUJARAT, INDIA,

]

WEB: www.indusuni.ac.in

NOV 2025

PROJECT REPORT

ON

Root ED

AT

ज्ञानेन प्रकाशते जगत्

INDUS UNIVERSITY

In the partial fulfillment of the requirement
for the degree of
Bachelor of Technology
in
Computer Science & Engineering

PREPARED BY

Gargie (IU2241230183)
Milee Bhatt (IU2241230225)
Dev Machchhar (IU2241230181)

UNDER GUIDANCE OF

Internal Guide
Mr.Hirenkumar Mer
Head of Department (i/c),
Department of Computer Science
&Engineering,I.T.E,
Indus University,Ahmedabad

SUBMITTED TO

INSTITUTE OF TECHNOLOGY AND ENGINEERING
INDUS UNIVERSITY CAMPUS, RANCHARDA, VIA-THALTEJ
AHMEDABAD-382115, GUJARAT, INDIA,
WEB: www.indusuni.ac.in

NOV 2025

CANDIDATE'S DECLARATION

I hereby declare that the Software Group Project report entitled "**Root Ed**" is the result of my own work carried out under the guidance and supervision of " **Mr**.**HIRENKUMAR MER**".

I further declare that, to the best of my knowledge, this report does not contain any part of the work that has been submitted previously for the award of any degree or diploma in this university or any other university, except where due acknowledgement and citation have been made.

Candidate's Signature

Milee Bhatt (IU2241230225)

Guide : Mr.Hirenkumar Mer Head of Department (i/c),
Department of Computer Science & Engineering,
Indus Institute of Technology and Engineering
INDUS UNIVERSITY– Ahmedabad,
State: Gujarat

CANDIDATE'S DECLARATION

I hereby declare that the Software Group Project report entitled "**Root Ed**" is the result of my own work carried out under the guidance and supervision of "**Mr.HIRENKUMAR MER**".

I further declare that, to the best of my knowledge, this report does not contain any part of the work that has been submitted previously for the award of any degree or diploma in this university or any other university, except where due acknowledgement and citation have been made.

Candidate's Signature

Gargie (IU2241230183)

Guide : Mr.Hirenkumar Mer Head of Department (i/c),
Department of Computer Science & Engineering,
Indus Institute of Technology and Engineering
INDUS UNIVERSITY– Ahmedabad,
State: Gujarat

CANDIDATE’S DECLARATION

I hereby declare that the Software Group Project report entitled “**Root Ed**” is the result of my own work carried out under the guidance and supervision of “**Mr.HIRENKUMAR MER**”.

I further declare that, to the best of my knowledge, this report does not contain any part of the work that has been submitted previously for the award of any degree or diploma in this university or any other university, except where due acknowledgement and citation have been made.

Candidate’s Signature

Dev Machchar (IU2241230181)

Guide : Mr.Hirenkumar Mer Head of Department (i/c),
Department of Computer Science & Engineering,
Indus Institute of Technology and Engineering
INDUS UNIVERSITY– Ahmedabad,
State: Gujarat

INDUS INSTITUTE OF TECHNOLOGY AND ENGINEERING

COMPUTER SCIENCE & ENGINEERING

NOV 2025

CERTIFICATE

Date:

This is to certify that the Software Group Project work entitled "**Root ED**" has been carried out by **Milee Bhatt(IU2241230225) , Gargie(IU2241230183) , DevMachchar(IU2241230181)** under my guidance in partial fulfillment of the requirements for the **Bachelor of Technology** in **Computer Science & Engineering (7th Semester)** of **Indus University, Ahmedabad** during the academic year 2025.

Mr.Hiren Mer
Head of Department (i/c),
Department of Computer Science & Engineering,
IITE, Indus University,
Ahmedabad

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to all those who have contributed to the successful completion of our project, **Root ED: AI-Driven Mathematics Education Platform**.

First and foremost, we would like to thank our guide**, Mr.Hirenkumar Mer**, for their continuous support, valuable guidance, and encouragement throughout the project. Their insights and suggestions helped us overcome challenges and refine our ideas.

We are also grateful to the **faculty members of INDUS University** for providing their expert knowledge, constructive feedback, and resources that greatly assisted our project development.

We would like to extend our heartfelt thanks to our **friends and group members**, whose teamwork, dedication, and cooperation were instrumental in completing this project. The collaboration and shared efforts made this project not only successful but also a rewarding learning experience.

Finally, we thank **INDUS University** for providing a conducive academic environment, facilities, and support throughout the duration of the project. We acknowledge and appreciate all the support and encouragement we have received from everyone involved.

Milee Bhatt (IU2241230225)
Dev Machchar (IU2241230181)
Gargie (IU2241230183)
B-Tech , Computer Science and Engineering
INDUS UNIVERSITY

TABLE OF CONTENT

ABSTRACT

The **Root ED: AI-Driven Mathematics Education Platform** is an innovative educational system designed to enhance the learning experience for students in mathematics. The platform leverages **Artificial Intelligence (AI)**, symbolic computation, and interactive visualization to provide **step-by-step solutions, explanations, and graphical representations** of mathematical problems.

Traditional mathematics learning methods often fail to provide personalized guidance or interactive visual understanding. Root ED addresses this gap by integrating **AI reasoning**, **Mathematical computation engines (SymPy, NumPy)**, and **dynamic visualization tools (Matplotlib, Plotly)** within a **user-friendly Streamlit-based interface**.

The system enables students to input problems from topics such as algebra, calculus, trigonometry, and statistics, and receive **detailed explanations**, **interactive plots**, and **history tracking** for improved comprehension. Additionally, Root ED is designed to support **educational institutions** by providing analytics, batch management, and administration features.

This project demonstrates how AI-powered platforms can revolutionize mathematics education by **bridging the gap between conceptual understanding and practical problem-solving**, while also offering scalability for future enhancements such as offline use, 3D visualizations, and gamified learning experiences.

Keywords: Artificial Intelligence, Mathematics Education, Interactive Visualization, Stepwise Explanation, Streamlit, SymPy, Plotly

List Of Figures

LIST OF TABLES

ABBREVIATION

Abbreviations used throughout this whole document for Survey Application are:

Abbreviation	**Description**
AI	Simulation of human intelligence in machines for problem-solving and learning
API	Interface that allows communication between different software components
GUI	Visual interface for user interaction with software
PDF	File format used for sharing formatted documents
ML	Subfield of AI focusing on learning from data
IDE	Software application providing comprehensive facilities to programmers
LaTeX	Typesetting system used for mathematical and scientific documentation
NLP	AI field dealing with interaction between computers and human languages
STEM	Educational approach integrating these fields
CSV	File format used for storing tabular data
JSON	Lightweight data interchange format
SQL	Language for managing relational databases
CPU	Hardware component that executes instructions
RAM	Temporary memory used by computers for active processes
URL	Address of a resource on the Internet
HTML	Standard markup language for web pages
CSS	Language used for describing the presentation of web pages
TCP	Core Internet protocol for reliable data transmission

Abbreviation	Description
UDP	Internet protocol for fast but unreliable transmission
IDE	Software suite for coding, testing, and debugging

CHAPTER 1

INTRODUCTION

- Project Summary
- Project Purpose
- Project Scope
- Objectives
- Technology and Literature Overview
- Synopsis

1.1 Project Summary

Mathematics is a subject that requires not only accurate answers but also a clear understanding of the underlying steps and concepts. Traditional digital tools such as calculators and symbolic solvers provide direct answers but lack conceptual explanations, leaving students confused about *why* a solution works.

To solve this gap, **Root ED** is developed as an **AI-powered Math Tutor** that combines artificial intelligence, symbolic computation, and visualization. The system provides **step-by-step explanations**, **LaTeX-formatted equations**, and **interactive graphs**, ensuring that learners understand concepts thoroughly rather than just memorizing results. The application is implemented using **Streamlit** for the frontend, **SymPy/NumPy** for mathematical computation, and **LangChain + GPT** for natural language explanations.

1.2 Project Purpose

The purpose of Root ED is to transform mathematics learning from an answer-centric process to a **concept-first approach**. It aims to:

- Provide a **personalized learning assistant** for students.
- Explain mathematical solutions in **clear and detailed steps**.
- Support **visual learning** through graphs and diagrams.
- Serve as a bridge between traditional tutoring and AI-driven EdTech solutions.

This project's ultimate goal is to help students develop deeper **mathematical intuition** while reducing the fear and complexity often associated with the subject.

1.3 Project Scope

Root ED has been designed with a broad scope to serve multiple user groups and purposes:

- **Target Users**: Students (school & college), self-learners, teachers, and EdTech platforms.
- **Problem Coverage**: Algebra, geometry, trigonometry, calculus, probability, and statistics.
- **Functional Scope**:
  - Accept mathematical queries in natural language or symbolic form.
  - Solve problems step by step with detailed explanations.
  - Generate LaTeX-formatted equations for neat presentation.
  - Display interactive **graphs** for equations and functions.
  - Provide a **chat-like interactive interface** with history and virtual math keyboard.
- **Exclusions**: Root ED is not a replacement for advanced research-level solvers but focuses on **educational and conceptual clarity** for students.

1.4 Objectives

1.4.1 Main Objectives

1. To build an **AI-powered interactive math tutor** that solves problems and explains them step by step.
2. To integrate **graphical visualization** for better conceptual clarity.
3. To use **LaTeX rendering** for professional mathematical formatting.
4. To create a **lightweight, user-friendly web app** using Streamlit.

1.4.2 Secondary Objectives

1. To provide **grade-specific explanations** for different learning levels.
2. To act as a **self-learning tool** for independent learners.
3. To serve as a foundation for **future integration** into EdTech platforms.
4. To encourage **interactive and visual learning** in mathematics.

1.5 Technology and Literature Overview

Technologies Used

- **Frontend**: Streamlit (for user interaction and UI design).
- **Backend AI**: OpenAI GPT + LangChain (for step-by-step natural language explanations).
- **Mathematical Engine**: SymPy, NumPy, and SciPy (for symbolic and numeric problem-solving).
- **Visualization Tools**: Matplotlib and Plotly (for graphs and plots).
- **Equation Rendering**: LaTeX integration in Streamlit (for clear formula presentation).

Literature Overview

Existing digital tools (e.g., calculators, WolframAlpha, and traditional e-learning apps) focus mainly on providing **solutions** rather than **explanations**. While symbolic computation systems can solve problems efficiently, they often lack **contextual explanations** tailored for learners. Recent advances in **Natural Language Processing (NLP)** and **Large Language Models (LLMs)** have opened opportunities to build systems that combine **computation + explanation + visualization**. Root ED leverages this synergy to create a **concept-first AI tutor** for mathematics.

1.6 Synopsis

The **Root ED Project** is an innovative AI-based solution aimed at revolutionizing math learning. By combining symbolic computation, artificial intelligence, and interactive visualization, it provides students with a **personalized tutoring experience**. Unlike conventional problem solvers, Root ED ensures students not only find the answer but also **understand the process** in detail. Its step-by-step breakdown, LaTeX-based formatting, and visual graphs make abstract concepts accessible and engaging.

Root ED contributes to the growing field of **AI in Education (AI-EdTech)** by promoting active learning, improving mathematical intuition, and helping students build strong problem-solving skills.

CHAPTER 2

LITERATURE SURVEY

- Introduction of Survey Scope
- Why Survey?
- Review of Existing Systems
- Gaps Identified from Literature
- Significance of Survey
- Conclusion of Literature Survey

2.1 Introduction of Survey

A literature survey is a critical step in any research or development project. It involves reviewing existing tools, systems, methodologies, and technologies that are related to the project, with the aim of identifying gaps, understanding trends, and drawing insights for improving the proposed solution.

In the context of **Root ED**, the literature survey focuses on existing **mathematical problem-solving platforms, AI-based educational tools, and interactive learning systems**. This study examines how previous approaches handle problem-solving, explanation generation, visualization, and user engagement.

The survey highlights that while many tools can provide correct answers, few offer **concept-based explanations** in a student-friendly manner combined with **interactive graphs and step-by-step reasoning**. This gap motivates the development of Root ED as a **concept-first, AI-powered math tutor**.

2.2 Why Survey?

Conducting a literature survey is essential for several reasons:

1. **Understanding Existing Solutions**:
   - Identify the capabilities and limitations of existing math tutoring systems like WolframAlpha, Symbolab, Photomath, and Khan Academy.
   - Learn what types of mathematical problems these systems can solve and how they explain solutions.
2. **Identifying Gaps**:

- Most existing platforms provide **answers without clear reasoning**, leaving students confused about the process.
- Visualization is often **limited or static**, lacking interactivity to aid conceptual understanding.
- Many tools are **general-purpose** and do not adjust explanations according to a student's **grade level or prior knowledge**.

3. **Informing System Design**:
    - Helps determine which **technologies, algorithms, and frameworks** can be integrated into the proposed system.
    - Provides insights for building a system that balances **computation, explanation, and visualization** effectively.
4. **Avoiding Redundancy**:
    - Ensures the proposed project **does not replicate existing solutions** but innovates by adding **interactive AI explanations** and concept-based learning.

2.3 Review of Existing Systems

2.3.1 WolframAlpha

- **Description**: An online computational engine that can solve equations, perform calculations, and generate basic graphs.
- **Strengths**: Accurate computation, wide coverage of mathematical topics.
- **Limitations**: Step-by-step explanations are minimal; visualizations are static and non-interactive; no customization based on learner's level.

2.3.2 Symbolab

- **Description**: Math solver providing step-by-step solutions for algebra, calculus, and other topics.

- **Strengths**: Good stepwise solution breakdown, user-friendly interface.
- **Limitations**: Explanations are textual only, graphs are limited, and advanced conceptual reasoning is lacking.

2.3.3 Photomath

- **Description**: Mobile app where users can scan math problems and get solutions.
- **Strengths**: Real-time solution recognition, simple explanations for basic problems.
- **Limitations**: Focuses mostly on arithmetic and algebra; limited interactive visualization; explanations are brief and non-customizable.

2.3.4 Khan Academy & Online Tutorials

- **Description**: Video-based teaching platform providing lessons and problem-solving examples.
- **Strengths**: Conceptual clarity through videos; covers a wide curriculum.
- **Limitations**: Non-interactive; students cannot input custom problems for immediate stepwise solutions; lacks real-time AI explanation.

2.4 Gaps Identified from Literature

- Most platforms provide either **answers** or **static explanations**, but not both **interactive explanations + visualizations**.
- Existing AI-based tools do not **personalize explanations** according to the learner's level.
- Few platforms combine **symbolic computation, AI reasoning, and graph visualization** in a single, lightweight web application.
- Real-time feedback for custom problems is **rarely available**, leaving students without immediate conceptual guidance.

These gaps justify the need for **Root ED**, which integrates **AI-generated step-by-step reasoning, LaTeX-rendered equations, interactive graphing, and a user-friendly Streamlit interface** to provide a **holistic math learning experience**.

2.5 Significance of Survey

- Provides a clear **understanding of the current landscape** of math learning tools.
- Helps in **designing Root ED's architecture and features** based on proven technologies while avoiding existing limitations.
- Supports the justification for **AI + visualization integration** in educational platforms.
- Guides future improvements and **ensures innovation** rather than repetition.

2.6 Conclusion of Literature Survey

The literature survey highlights that while numerous digital tools exist for solving mathematical problems, none fully meet the combined requirements of **interactive visualization, step-by-step reasoning, concept-based explanations, and AI-assisted tutoring**.

Root ED addresses these gaps by:

1. Providing **step-by-step AI-generated explanations** tailored to the learner.
2. Integrating **interactive graphs and visualizations** to enhance conceptual understanding.

3. Offering a **lightweight and accessible web application** interface through Streamlit.

This survey confirms the **novelty and necessity of Root ED** as a comprehensive solution for improving math education

CHAPTER 3

PROJECT MANAGEMENT

- Project Planning Objectives
- Project Scheduling
- Risk Management

3.1 Project Planning Objectives

Project planning is crucial to ensure that Root ED is developed efficiently, meets quality standards, and is delivered on time. The planning objectives include:

- Defining clear project scope and deliverables.
- Identifying required resources, including human, software, and environment.
- Determining an appropriate development approach for timely implementation.
- Establishing a project schedule and risk management framework.

3.1.1 Software Scope

The software scope defines the boundaries and functionalities of Root ED:

- **Functional Scope**:
    - Accept mathematical queries in text or equation form.
    - Provide **step-by-step AI-generated explanations**.
    - Render **LaTeX-formatted equations** for clarity.
    - Generate **interactive graphs** for visualization.
    - Maintain **chat-like interface with history and virtual keyboard**.
- **Non-functional Scope**:
    - User-friendly interface.
    - Fast response time for queries.
    - Scalable architecture for future feature expansion.

3.1.2 Resource

Successful development requires proper allocation of **human, software, and environment resources**.

3.1.2.1 Human Resource

- **Project Manager**: Oversees timelines, deliverables, and coordination.
- **AI/ML Engineer**: Integrates GPT models, LangChain, and handles AI reasoning.
- **Frontend Developer**: Designs Streamlit interface, virtual keyboard, and interactive graphs.
- **Backend Developer**: Handles SymPy, NumPy, and computation modules.
- **Tester/QA**: Ensures accuracy of calculations, explanations, and UI functionality.

3.1.2.2 Reusable Software Resources

- **Python Libraries**: SymPy, NumPy, SciPy, Matplotlib, Plotly.
- **AI Tools**: OpenAI GPT, LangChain.
- **Visualization Tools**: LaTeX integration for equations, Streamlit for UI.

- **Code Templates/Modules**: Pre-existing Streamlit components and plotting functions.

3.1.2.3 Environment Resource

- **Development Environment**: Python IDE (PyCharm/VS Code), Streamlit server.
- **Testing Environment**: Local machines and cloud-based test instances.
- **Version Control**: GitHub for source code management.
- **Hardware**: Standard PC with internet access, GPU optional for AI computations.

3.1.3 Project Development Approach

The **Agile Development Methodology** is adopted for Root ED:

- Iterative development with frequent testing.
- Flexibility to incorporate user feedback during development.
- Focus on delivering **incremental modules** (computation engine, AI explanations, visualization) in short sprints

3.2 Project Scheduling

Project scheduling ensures tasks are allocated properly, dependencies are managed, and milestones are met.

3.2.1 Basic Principles

- Define all tasks and milestones.
- Assign resources and timelines for each task.
- Track progress regularly to avoid delays.

3.2.2 Compartmentalization

Project tasks are divided into manageable modules:

1. Requirement Gathering & Analysis
2. System Design (UI, computation, AI explanation flow)
3. Backend Development (SymPy, NumPy modules)
4. AI Integration (GPT + LangChain)
5. Visualization & Graphing
6. Testing & QA
7. Deployment & Documentation

3.2.3 Work Breakdown Structure (WBS)

TABLE NO.1

WBS ID	Task Description	Duration	Responsible
1	Requirement Analysis	1 week	Project Manager
2	System Design	1 week	Frontend & Backend Developers
3	Backend Module Development	2 weeks	Backend Developer
4	AI Integration	2 weeks	AI/ML Engineer
5	Visualization & UI	1 week	Frontend Developer
6	Testing & QA	1 week	Tester
7	Deployment & Documentation	1 week	All Team Members

3.2.4 Project Organization

- **Team Structure**: Hierarchical with Project Manager at top, coordinating frontend, backend, AI, and testing teams.
- **Communication**: Weekly meetings, Slack/GitHub updates, task tracking via Trello or Jira.

3.2.5 Timeline Chart

3.2.5.1 Time Allocation

- Total project duration: **9 weeks**

- Sprints: 1-2 weeks per module as per WBS.

3.2.5.2 Task Sets

TABLE NO. 2

Week	Task
1	Requirement Analysis
2	System Design
3-4	Backend Module Development
5-6	AI Integration
7	Visualization & UI
8	Testing & QA
9	Deployment & Documentation

3.3 Risk Management

Risk management identifies, analyzes, and mitigates potential issues that may affect project success.

3.3.1 Risk Identification

- Delays in AI model integration due to technical complexity.
- Errors in symbolic computation or visualization.
- UI/UX not meeting user-friendly standards.
- Server or performance issues for Streamlit deployment.

3.3.1.1 Risk Identification Artifacts

- **Risk Register**: Document containing all identified risks, probability, impact, and mitigation plan.
- **Issue Tracker**: Maintained on GitHub/Trello to monitor ongoing risks and resolutions.

3.3.2 Risk Projection

TABLE NO.3

Risk	**Probability**	**Impact**	**Mitigation Strategy**
AI integration delays	Medium	High	Start with small test models; maintain fallback manual explanations
Computation errors	Low	High	Unit testing of SymPy/NumPy modules
UI/UX issues	Medium	Medium	Conduct user testing with feedback
Deployment/performance issues	Low	Medium	Optimize code; use cloud deployment if needed

CHAPTER 4

SYSTEM ANALYSIS & DESIGN

- **Problem Definition**
- **Existing System**
- **Proposed System**
- **Feasibility Study**
- **Data Flow Diagrams (DFD)**
- **System Architecture**
- **Conclusion**

4.1 Problem Definition

Mathematics is a subject that requires logical reasoning and stepwise problem-solving. Traditional tools provide **answers** but do not focus on **conceptual understanding**. Students often struggle with:

- Understanding **how solutions are derived**.
- Visualizing abstract mathematical concepts (functions, equations, calculus graphs).
- Accessing **interactive, personalized learning** at their convenience.

The **problem** is the absence of an AI-driven platform that **combines step-by-step explanations, visualization, and interactive learning** in a single lightweight application. Root ED addresses this gap.

4.2 Existing System

Existing systems include:

1. **Calculators & Solvers (WolframAlpha, Symbolab)**
   - Pros: Accurate solutions, basic stepwise explanations.
   - Cons: Limited interactivity; explanations not tailored to learner level.
2. **Mobile Apps (Photomath)**
   - Pros: Quick recognition and solution of scanned problems.
   - Cons: Minimal explanations; visualizations are static; limited topics coverage.
3. **Online Tutorials (Khan Academy, YouTube)**
   - Pros: Concept explanation through videos.
   - Cons: Non-interactive; learners cannot input custom problems for immediate feedback.

Gap: None of these provide **AI-generated, stepwise explanations with interactive visualizations** personalized to the learner's level.

4.3 Proposed System

Root ED is proposed to overcome these limitations by providing:

- **Step-by-step AI explanations** using GPT models.
- **LaTeX-rendered equations** for clarity.
- **Interactive graphs and plots** for visualization.
- **Streamlit-based web interface** with chat-like input and virtual math keyboard.
- **Personalized explanation levels** based on learner grade.

4.3.1 Key Features

1. Problem input in text or symbolic form.
2. Automated solution computation using **SymPy & NumPy**.
3. AI-generated **explanation breakdown** via LangChain + GPT.
4. Graphs for better concept understanding using **Matplotlib/Plotly**.
5. History and interactive interface for easy navigation.

4.4 Feasibility Study

4.4.1 Technical Feasibility

- Languages/Tools: Python, Streamlit, SymPy, Matplotlib, Plotly.
- AI Integration: OpenAI GPT + LangChain.
- Hardware Requirements: Standard PC; GPU optional for AI computations.
- Conclusion: Technically feasible; libraries and frameworks are available.

4.4.2 Economic Feasibility

- Open-source libraries reduce software cost.
- Streamlit deployment on free or low-cost cloud servers minimizes hardware expense.
- Conclusion: Economically feasible for student or educational use.

4.4.3 Operational Feasibility

- Easy-to-use interface ensures user adoption.
- Can be used independently by students or integrated into EdTech platforms.
- Conclusion: Operationally feasible; aligns with educational goals.

4.5 Data Flow Diagrams (DFD)

4.5.1 Level 0 DFD (Context Diagram)

Student ---> Root ED System ---> Solution + Explanation + Graphs

4.5.2 Level 1 DFD

1. **User Input Module**: Accepts math problem.
2. **Preprocessing Module**: Detects problem type.
3. **Computation Engine**: Calculates solution using SymPy/NumPy.
4. **AI Explanation Module**: GPT generates stepwise explanation.
5. **Visualization Module**: Generates interactive graphs.
6. **Output Module**: Displays solution, explanation, and graphs via Streamlit.

4.6 System Architecture

Proposed Architecture: FIGURE NO.1

```
+-------------------+      +--------------------+
|  User Interface  |<--->|  Streamlit Frontend|
+-------------------+      +--------------------+
       |                        |
       v                        v
+-------------------+      +--------------------+
|  Preprocessing    |      | Computation Engine |
|  (NLP, Parser)    |      | SymPy, NumPy       |
+-------------------+      +--------------------+
       |                        |
       +------------+----------------+
                 |
                 v
           +----------------+
           | AI Explanation |
           | GPT + LangChain|
           +----------------+
                 |
                 v
           +----------------+
           | Visualization  |
           | Matplotlib/Plotly|
           +----------------+
                 |
                 v
           +----------------+
           | Output Display |
           | Streamlit UI   |
           +----------------+
```

4.7 Conclusion

The **Root ED system design** addresses the limitations of existing math learning platforms by combining **symbolic computation, AI reasoning, interactive visualization, and user-friendly interface**. It is **technically, economically, and operationally feasible**, providing a comprehensive AI-powered learning experience for students.

CHAPTER 5

SYSTEM ANALYSIS

- **Study of Current System**
- Problems in Current System
- Requirement of New System
- Process Model
- Feasibility Study
- Features of New System

5.1 Study of Current System

The current systems for solving and learning mathematics include:

1. **Traditional Calculators & Solvers**
   - Provide direct answers to mathematical problems.
   - Limited to numeric computation; symbolic reasoning is minimal.
2. **Online Platforms (WolframAlpha, Symbolab, Photomath)**
   - Pros: Solve algebra, calculus, and other math problems efficiently.
   - Cons: Explanations are brief; graphs and visualizations are static or limited.
3. **Video Tutorials & e-Learning Platforms (Khan Academy, YouTube)**
   - Pros: Teach concepts with detailed video lessons.
   - Cons: Non-interactive; students cannot input custom problems for instant guidance.

Observation: Existing systems either focus on **solutions only** or **concept explanation only**. None combine **interactive problem solving, AI-generated stepwise explanations, and visualization** in one lightweight platform.

5.2 Problems in Current System

1. Lack of **step-by-step explanations** tailored to different learner levels.
2. Absence of **interactive visualization** for abstract math concepts.
3. Limited **personalization**: users cannot adjust explanation depth.
4. Non-interactive interfaces prevent **real-time feedback**.
5. High reliance on **manual learning** for conceptual understanding.
6. Deployment issues: some platforms are mobile-only or require subscription.

5.3 Requirement of New System

The proposed system, **Root ED**, addresses the above issues by:

- Providing **AI-driven stepwise explanations** for every math problem.
- Offering **LaTeX-rendered equations** for clean and readable formulas.
- Generating **interactive graphs and plots** to improve concept clarity.
- Using a **user-friendly Streamlit interface** with chat history and virtual keyboard.
- Supporting a **wide range of topics**: algebra, calculus, trigonometry, statistics, and probability.
- Allowing **customized learning levels** based on student grade or prior knowledge.

5.4 Process Model

The **Agile Development Process** is adopted for Root ED:

1. **Requirement Analysis** – Identify project objectives, user needs, and functional specifications.
2. **Design** – Create system architecture, module design, and interface prototypes.
3. **Implementation** – Develop modules:
   - User input and preprocessing
   - Computation engine (SymPy, NumPy)
   - AI explanation module (GPT + LangChain)
   - Visualization module (Matplotlib/Plotly)
   - Frontend integration (Streamlit)
4. **Testing** – Unit testing, integration testing, and user acceptance testing.
5. **Deployment & Feedback** – Launch web application and collect user feedback for improvements.
6. **Iteration** – Refine system based on testing and feedback in short sprints.

5.5 Feasibility Study

5.5.1 Technical Feasibility

- **Technologies Used**: Python, Streamlit, SymPy, NumPy, Matplotlib, Plotly, OpenAI GPT, LangChain.
- **Reasoning**: All technologies are open-source or accessible, making development straightforward.
- **Hardware Requirements**: Standard PC; optional GPU for faster AI computation.
- **Conclusion**: Technically feasible; resources are available and compatible.

5.5.2 Operational Feasibility

- **Ease of Use**: Streamlit ensures a simple and interactive web interface.
- **User Adoption**: Target users (students, teachers) can easily interact with the system.
- **Maintenance**: Lightweight architecture allows easy updates and bug fixes.
- **Conclusion**: Operationally feasible; aligns with user needs and educational goals.

5.5.3 Economical Feasibility

- **Cost Efficiency**: Uses free or open-source libraries; minimal hardware cost.
- **Deployment Cost**: Can be deployed on low-cost cloud servers.

- **Conclusion**: Economically viable; low development and maintenance costs.

5.5.4 Schedule Feasibility

- Project duration: **9 weeks**, with defined milestones per module.
- Agile methodology ensures **incremental delivery** and flexibility in timeline.
- Conclusion: Schedule is feasible and realistic for completion.

5.5 Features of New System

1. **Step-by-Step AI Explanations** – Detailed reasoning for each solution.
2. **Interactive Visualization** – Graphs for functions, equations, and geometry.
3. **LaTeX Equation Rendering** – Professional and clear display of formulas.
4. **User-Friendly Interface** – Chat-like input with virtual keyboard and history.
5. **Wide Topic Coverage** – Algebra, calculus, trigonometry, probability, and statistics.
6. **Personalized Learning Levels** – Tailors explanations to the student's knowledge.
7. **Lightweight and Accessible** – Streamlit ensures cross-platform usability with minimal setup.

CHAPTER 6

DETAILED DESCRIPTION

- **User Module**
- **Company/Institution Module *(adapted for EdTech platforms or classrooms)***
- **Administrator Module**

6.1 User Module

The **User Module** represents the primary students or learners interacting with the Root ED system. This module is responsible for providing an interface where users can input math problems, receive solutions, and view explanations.

Key Features:

1. **User Input Interface**
    - Users can type problems in natural language or symbolic form.
    - Virtual math keyboard for easier input of equations and symbols.
2. **Problem Processing**
    - Preprocessing of input to identify problem type (algebra, calculus, statistics, etc.).
    - Validation of input to ensure correctness before computation.
3. **AI Explanation Module**
    - Generates **step-by-step reasoning** for each solution using GPT + LangChain.
    - Provides explanations tailored to the learner's grade or understanding level.
4. **Visualization**
    - Displays **interactive graphs, plots, and diagrams** using Matplotlib/Plotly.
    - Helps learners understand abstract concepts visually.
5. **Output & History**
    - Displays solutions in **LaTeX format** for clarity.
    - Maintains a **history of previous queries and solutions** for review.

Benefits:

- Students gain **conceptual understanding**, not just answers.

- Real-time feedback improves learning efficiency.
- Interactive visualizations make complex topics easier to grasp.

6.2 Company/Institution Module *(adapted for EdTech platforms or classrooms)*

The **Company Module** represents educational institutions, online learning platforms, or EdTech partners that may use Root ED for student learning management.

Key Features:

1. **Batch Access & Management**
   - Institutions can create groups or classes of users.
   - Assign specific topics or problem sets to batches.
2. **Monitoring & Analytics**
   - Track student usage and performance.
   - Generate reports on frequently asked topics or areas where students struggle.
3. **Resource Sharing**
   - Share pre-defined problem sets, exercises, and learning resources within the system.
4. **Integration**
   - Integrate Root ED with existing e-learning platforms or Learning Management Systems (LMS).

Benefits:

- Facilitates **institutional learning** with AI-powered tutoring.
- Helps teachers identify **weak areas** in students' understanding.
- Promotes **personalized learning** at scale.

6.3 Administrator Module

The **Administrator Module** handles system configuration, user management, and overall maintenance.

Key Features:

1. **User Management**
   - Add, update, or delete user accounts.
   - Assign roles and permissions (student, teacher, institution).
2. **System Configuration**
   - Manage AI model settings, explanation depth, and computation resources.
   - Control visualization features and user interface settings.
3. **Monitoring & Reports**
   - Track system performance and usage statistics.
   - Generate analytics for administrative decision-making.
4. **Security & Access Control**
   - Ensure secure authentication and authorization.
   - Maintain data privacy for users and institutions.

Benefits:

- Ensures **smooth system operation** and resource optimization.
- Maintains **data integrity and security**.
- Supports **scalable deployment** for multiple users or institutions.

Conclusion

The **Detailed Description** chapter outlines how Root ED caters to **different types of users**:

- **Students** interact with the system to learn and solve math problems.

- **Institutions** use it for monitoring and supporting student learning.
- **Administrators** maintain system functionality, security, and performance.

This modular approach ensures that Root ED is **scalable, user-friendly, and effective** in delivering AI-powered math education.

CHAPTER 7

TESTING

- **Black-Box Testing**
- **White-Box Testing**
- **Test Cases**

7.1 Black-Box Testing

Black-box testing focuses on testing the functionality of the Root ED system without knowing the internal code structure. The objective is to ensure the system meets its functional requirements.
Key Testing Areas:

1. Problem Input Validation
   - Entering valid and invalid mathematical queries.
   - Ensuring the system handles errors gracefully.
2. Step-by-Step Explanation Generation
   - Verify that AI-generated explanations are accurate and coherent.
   - Check consistency for similar problem types.
3. Graphical Visualization
   - Confirm that graphs/plots render correctly for various equations.
   - Test interactive features like zoom, pan, and plot updates.
4. Output Formatting
   - Verify LaTeX rendering of equations and solution formatting.
5. User Interface & Navigation
   - Ensure chat history works correctly.
   - Virtual keyboard inputs should be properly recognized.

Objective: Validate that all functional requirements are met and the system behaves as expected for end-users.

7.2 White-Box Testing

White-box testing examines the internal structure and logic of the Root ED system. This ensures that all modules, functions, and algorithms perform correctly.
Key Testing Areas:

1. Computation Engine Testing
   - Verify SymPy and NumPy calculations for accuracy.
   - Check corner cases (e.g., division by zero, undefined functions).
2. AI Explanation Module Testing
   - Validate reasoning logic and proper flow in generating stepwise solutions.
   - Ensure all branches of problem types are handled.
3. Visualization Module Testing
   - Ensure plotting functions execute without errors.
   - Validate correct mapping of mathematical functions to graphs.
4. Integration Testing

   - Test the seamless flow between input, computation, AI explanation, visualization, and output modules.
5. Code Coverage Analysis
   - Ensure all functions, loops, and conditional statements are tested.
   - Identify and fix dead code or untested modules.

Objective: Ensure internal logic and modules work correctly and no functional flaws exist at the code level.

7.3 Test Cases

TABLE NO.4

S.No	Test Case Description	Input	Expected Output	Actual Output	Status
1	Solve algebraic equation	x^2 - 5x + 6 = 0	x = 2, x = 3 (step-by-step)	Correct solution with steps	Pass
2	Graph a quadratic function	y = x^2 - 4x + 3	Plot showing parabola with vertex at (2, -1)	Correct interactive graph	Pass
3	Invalid input handling	x^^2 + 5	Error message	Displays "Invalid input"	Pass
4	Trigonometric problem	sin(x) + cos(x) = 1	Stepwise solution & graph	Correct solution & plot	Pass
5	History feature check	Multiple queries	Display previous queries with solutions	History maintained	Pass
6	LaTeX rendering test	Integral of x^2	Properly formatted equation display	Correct rendering	Pass
7	Division by zero	1/(x-2) when x=2	Error handling	Displays "Undefined"	Pass

CHAPTER 8

SYSTEM DESIGN

- **Class Diagram**
- **Use-Case Diagram**
- **Sequence Diagram**
- **Activity Diagram**
- **Data Flow Diagram (DFD)**

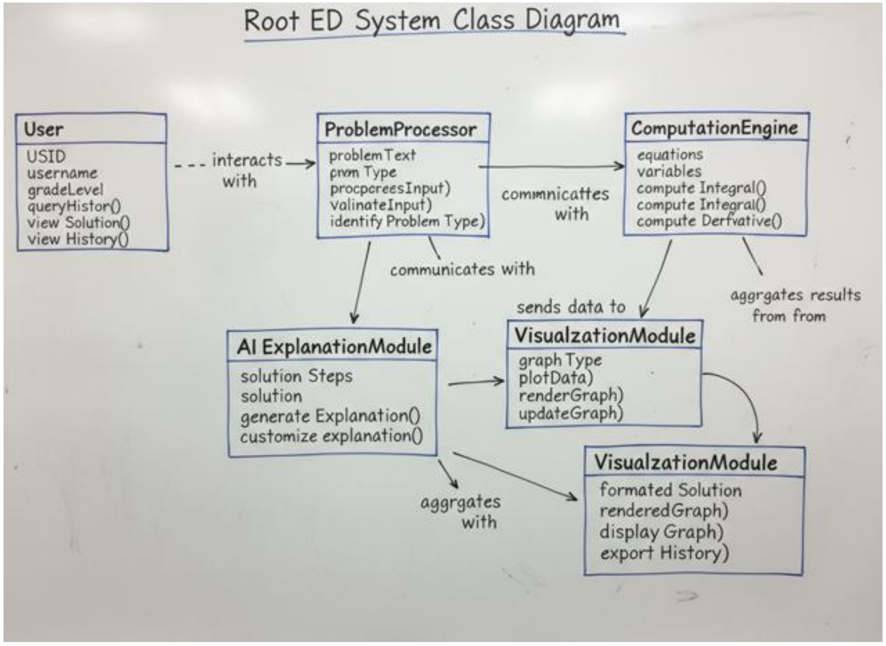
8.1 Class Diagram

The Class Diagram represents the structure of the Root ED system, showing classes, attributes, methods, and relationships.
Key Classes:

1. User
   - Attributes: userID, username, gradeLevel, queryHistory
   - Methods: submitQuery(), viewSolution(), viewHistory()
2. ProblemProcessor
   - Attributes: problemText, problemType
   - Methods: preprocessInput(), validateInput(), identifyProblemType()
3. ComputationEngine
   - Attributes: equations, variables
   - Methods: solveEquation(), computeIntegral(), computeDerivative()
4. AIExplanationModule
   - Attributes: solutionSteps, explanationLevel
   - Methods: generateExplanation(), customizeExplanation()
5. VisualizationModule
   - Attributes: graphType, plotData
   - Methods: generateGraph(), renderGraph(), updateGraph()
6. OutputDisplay
   - Attributes: formattedSolution, renderedGraph
   - Methods: displaySolution(), displayGraph(), exportHistory()

Relationships:

- User interacts with ProblemProcessor and OutputDisplay.
- ProblemProcessor communicates with ComputationEngine and AIExplanationModule.
- AIExplanationModule sends data to VisualizationModule.
- OutputDisplay aggregates results from ComputationEngine, AIExplanationModule, and VisualizationModule.
  FIGURE NO.2



8.2 Use-Case Diagram

The Use-Case Diagram shows the interactions between users and the Root ED system.

Actors:

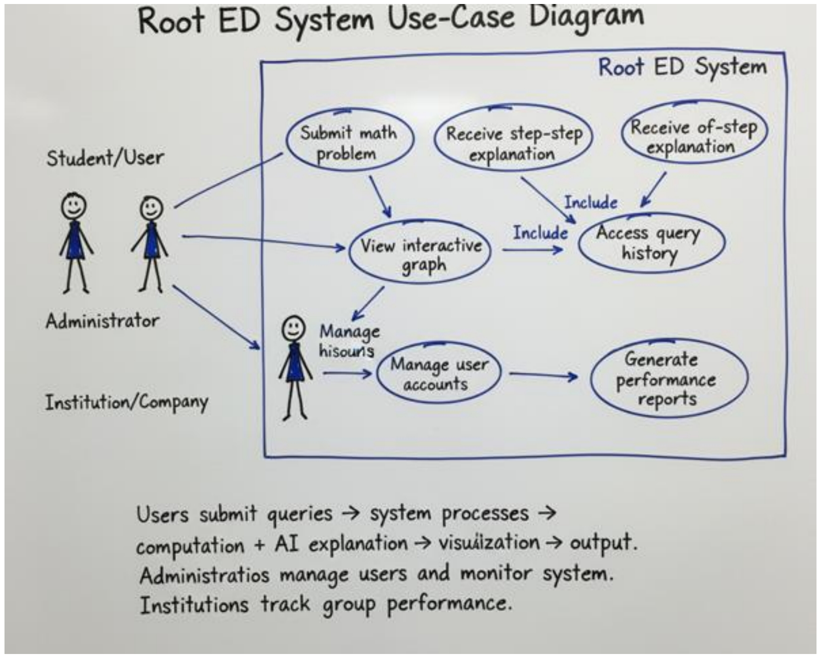
- Student/User
- Administrator
- Institution/Company

Use Cases:

1. Submit math problem
2. Receive step-by-step explanation
3. View interactive graph
4. Access query history
5. Manage user accounts (Admin)
6. Generate performance reports (Institution)

Description:

- Users submit queries → system processes → computation + AI explanation → visualization → output.
- Administrators manage users and monitor system.
- Institutions track group performance.
  FIGURE NO.3

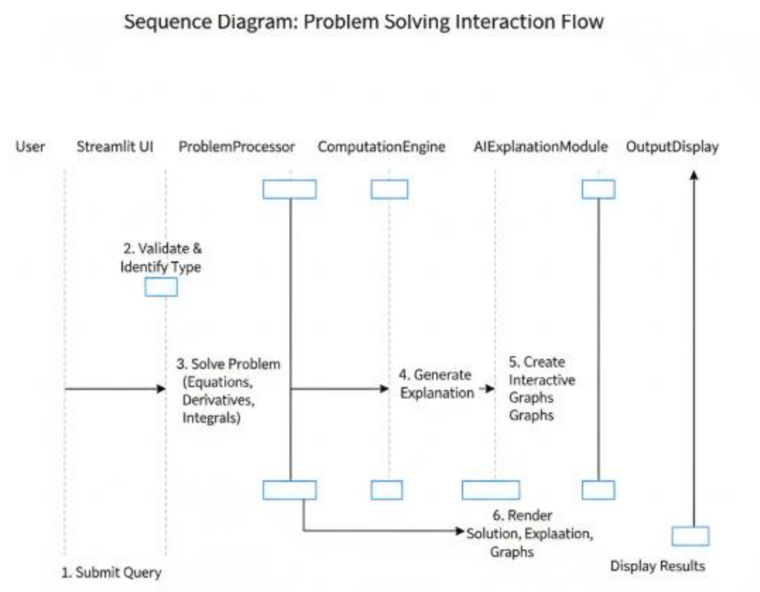


8.3 Sequence Diagram

The Sequence Diagram represents the interaction flow for solving a problem.
Steps:

1. User submits a query via Streamlit interface.
2. ProblemProcessor validates and identifies problem type.
3. ComputationEngine solves the problem (equations, derivatives, integrals).
4. AIExplanationModule generates step-by-step explanation.
5. VisualizationModule creates interactive graphs.
6. OutputDisplay renders solution, explanation, and graphs for the user.

Flow Representation:
User → Streamlit UI → ProblemProcessor → ComputationEngine → AIExplanationModule → VisualizationModule → OutputDisplay → User
FIGURE NO.4



8.4 Activity Diagram

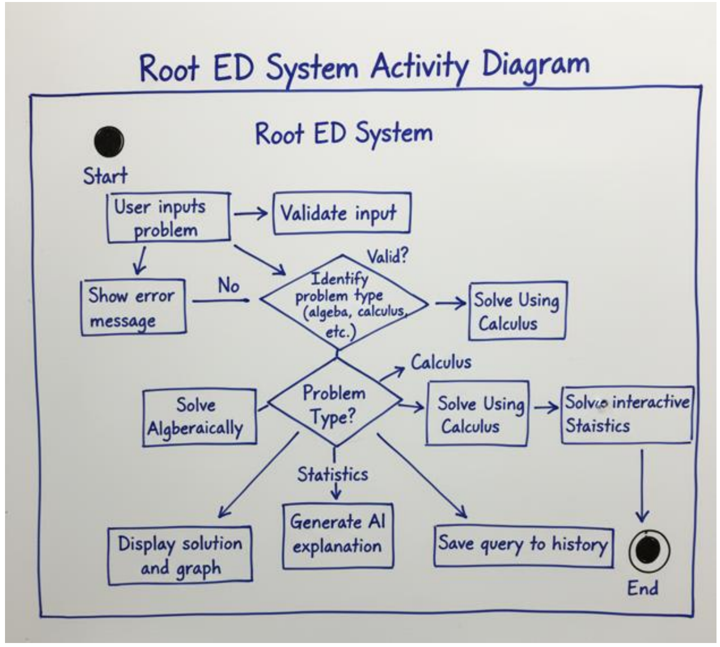
The Activity Diagram describes the workflow of the Root ED system.
Steps:

1. User inputs problem
2. Validate input → if invalid, show error → else continue
3. Identify problem type (algebra, calculus, etc.)
4. Solve problem using ComputationEngine
5. Generate AI explanation
6. Generate interactive graph
7. Display solution and graph
8. Save query to history
9. End

Decision Points:

- Input validation: valid vs invalid
- Problem type identification: algebra vs calculus vs statistics

FIGURE NO.5



8.5 Data Flow Diagram (DFD)

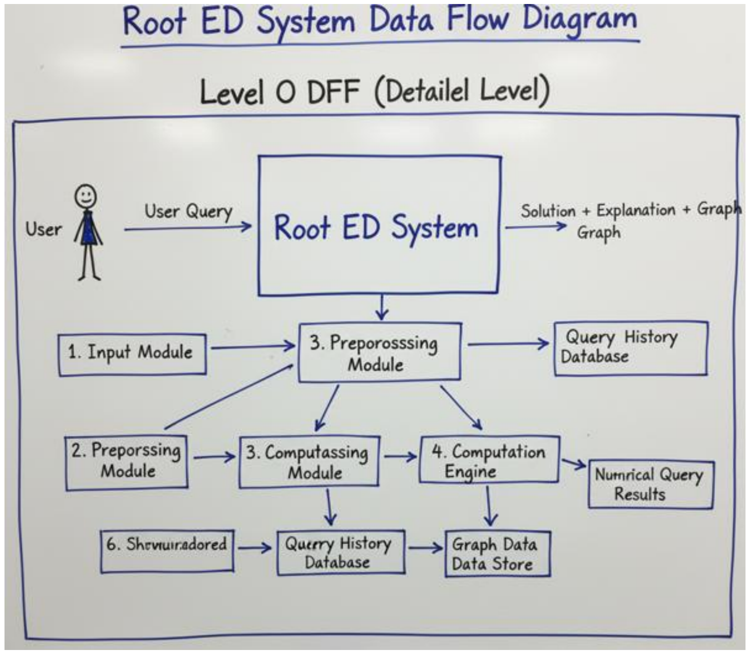
The DFD represents data movement within Root ED.
Level 0 DFD (Context Level):
User → Root ED System → Solution + Explanation + Graph
Level 1 DFD (Detailed):

1. Input Module: Receives user query
2. Preprocessing Module: Validates and categorizes problem
3. Computation Engine: Performs mathematical calculations
4. AI Explanation Module: Generates stepwise solution
5. Visualization Module: Creates interactive graphs
6. Output Module: Displays results and saves history

Data Stores:

- Query History Database: Stores all user queries and solutions
- Graph Data Store: Stores visualization data for interactive plots

FIGURE NO.


Root ED System Data Flow Diagram
Level 0 DFF (Detailel Level)
User
User Query
Root ED System
Solution + Explanation + Graph
Graph
1. Input Module
3. Preporosssing Module
Query History Database
2. Preporssing Module
3. Computassing Module
4. Computation Engine
Numrical Query Results
6. Shevuuiradored
Querry History Database
Graph Data Data Store

CHAPTER 9

LIMITATION AND FUTURE ENHANCEMENT

- **Limitation**
- **Future Enhancement**

9.1 Limitation

While **Root ED** is a powerful AI-driven math learning platform, there are some limitations:

1. **Dependency on Internet and AI Models**
   - Requires internet connectivity to access GPT-based AI explanations.
   - Performance depends on AI model response times and availability.
2. **Limited Problem Types**
   - Currently optimized for algebra, calculus, trigonometry, statistics, and probability.
   - May not handle highly advanced or domain-specific mathematics topics.
3. **Graph Complexity**
   - Visualization is limited to 2D/3D plots supported by Matplotlib/Plotly.
   - Complex visualizations like dynamic 3D surfaces or animations are not yet implemented.
4. **Scalability for Large User Base**
   - Streamlit deployment is ideal for small-to-medium usage.
   - High concurrent users may require cloud deployment and load balancing.
5. **AI Explanation Limitations**
   - GPT-generated explanations may occasionally be **ambiguous** or require manual verification.
   - Step-by-step reasoning may not perfectly align with all teaching methods.

9.2 Future Enhancement

The system can be enhanced in multiple ways to make Root ED more robust and comprehensive:

1. **Offline Functionality**
    - Implement local AI inference for offline usage.
    - Precompute commonly used solutions to reduce dependency on internet.
2. **Expanded Problem Coverage**
    - Support advanced topics such as linear algebra, differential equations, combinatorics, and graph theory.
    - Integrate domain-specific modules for engineering, physics, or finance mathematics.
3. **Advanced Visualization**
    - Incorporate dynamic 3D plotting and animations for better conceptual understanding.
    - Use interactive visualizations with sliders, animation, and real-time updates.
4. **Mobile Application**
    - Develop a cross-platform mobile app for iOS and Android.
    - Include features like camera-based problem input using OCR.
5. **Personalized Learning Paths**
    - AI-based recommendations for learning sequences based on student performance.
    - Adaptive difficulty adjustment to match learner's pace.
6. **Cloud Deployment & Scalability**

- Deploy Root ED on cloud platforms (AWS, Azure, GCP) for high availability and performance.
- Support multiple concurrent users with load balancing and database optimization.

7. **Gamification & Progress Tracking**
    - Include points, badges, and quizzes to motivate learners.
    - Provide detailed performance analytics and improvement suggestions.

CHAPTER 10

CONCLUSION

- **Conclusion**

10.1 Conclusion

The **Root ED project** successfully demonstrates the integration of **AI-powered reasoning, symbolic computation, and interactive visualization** to enhance the learning of mathematics. The system effectively addresses the limitations of traditional learning tools by providing:

- **Step-by-step explanations** generated using GPT models, enabling students to understand the reasoning behind solutions.
- **Interactive graphing and visualizations** that make abstract mathematical concepts easier to comprehend.
- **User-friendly interface** through Streamlit, including chat history and virtual math keyboard for seamless interaction.
- **Scalable and modular architecture** supporting future enhancements, institutional usage, and personalized learning.

The development process, including **requirement analysis, system design, implementation, and rigorous testing**, ensures that Root ED is reliable, accurate, and effective as an educational tool. While some limitations exist, such as dependency on AI services, limited advanced topic coverage, and scalability constraints, the **proposed future enhancements** provide a roadmap to overcome these challenges and extend the system's capabilities.

Final Remarks:
Root ED represents a **significant step toward intelligent, interactive, and personalized mathematics education**, combining computation, AI reasoning, and visualization in a single platform. The project highlights the potential of AI-driven educational tools to transform the learning experience and lays a foundation for continuous improvement and expansion in the future.

CHAPTER 11

APPENDICES

- **Business Model**
- **Product Deployment Detail**
- **API and Web Service Details**

11.1 Business Model

Root ED can be positioned as an **AI-driven educational platform** for students, schools, and online learning platforms. The business model includes:

1. **Target Audience:**
   - Students (High School to College level)
   - Educational institutions (schools, colleges, coaching centers)
   - EdTech companies seeking AI-enhanced learning tools
2. **Revenue Streams:**
   - **Freemium Model:** Basic problem-solving and explanations free; premium access includes advanced topics, personalized learning paths, and additional visualization tools.
   - **Subscription Plans:** Monthly or annual subscription for schools and institutions with multiple user licenses.
   - **Enterprise Licensing:** For EdTech platforms integrating Root ED into their system.
   - **Adoption in Online Courses:** Integration with MOOCs or online tutorials with branded AI learning.
3. **Value Proposition:**
   - Personalized AI explanations tailored to student level

- Interactive visualization of mathematical concepts
- Efficient learning for both individual and institutional users

11.2 Product Deployment Detail

Deployment Environment:

- **Frontend:** Streamlit web interface
- **Backend:** Python-based computation and AI modules
- **Visualization:** Matplotlib, Plotly for interactive graphs
- **AI Services:** GPT models via LangChain/OpenAI API
- **Database:** SQLite or cloud-based DB for storing query history

Deployment Steps:

1. Host the Streamlit app on a local server or cloud platform (Heroku, AWS, or Azure).
2. Configure AI service API keys and ensure secure connectivity.
3. Set up database for user management and query history storage.
4. Integrate computation engine (SymPy/NumPy) and visualization modules.
5. Test all modules in staging environment before public deployment.
6. Continuous monitoring and maintenance for uptime and performance.

Access:

- Web-based access via browser (cross-platform: Windows, Mac, Linux).

- Optional future deployment as a mobile app for Android/iOS

11.3 API and Web Service Details

APIs Used:

1. **OpenAI GPT API**
   - Purpose: Generate step-by-step explanations for math problems.
   - Input: Problem text, user grade level, explanation depth.
   - Output: AI-generated solution steps in text format.
2. **SymPy Library (Python)**
   - Purpose: Symbolic computation of equations, derivatives, integrals.
   - Input: Parsed mathematical expressions.
   - Output: Computed solutions in symbolic or numeric format.
3. **NumPy Library (Python)**
   - Purpose: Perform numeric calculations and array operations for visualization.
4. **Matplotlib / Plotly API**
   - Purpose: Generate 2D/3D interactive graphs of mathematical functions.

- Input: Computed function values or coordinates.
- Output: Interactive plot displayed in Streamlit interface.

Web Services:

- Streamlit web service provides **real-time interaction** with users.
- Handles user input, calls computation and AI modules, renders outputs, and displays visualization.
- Secure API connections with OpenAI ensure privacy and reliability.

BIBLOGRAPHY

1. **Books and Research Papers**
   - Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
   - SymPy Development Team. (2024). *SymPy Documentation: Symbolic Mathematics in Python*. https://www.sympy.org
   - Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th Edition). Pearson.
   - Mitra, S. (2019). *AI in Education: Transforming Learning with Artificial Intelligence*. Springer.
2. **Websites and Documentation**
   - Streamlit Official Documentation. https://docs.streamlit.io
   - OpenAI API Documentation. https://platform.openai.com/docs
   - Matplotlib Documentation. https://matplotlib.org/stable/contents.html
   - Plotly Documentation. https://plotly.com/python/
   - NumPy Documentation. https://numpy.org/doc/stable/
3. **Journals and Articles**

- Chen, L., Chen, P., & Lin, Z. (2020). *Artificial Intelligence in Education: A Review*. IEEE Access, 8, 75264–75278.
- Heffernan, N., & Heffernan, C. (2014). *The ASSISTments Ecosystem: Building a Platform that Brings Scientists and Teachers Together for Minimally Invasive Research on Human Learning and Teaching*. International Journal of Artificial Intelligence in Education, 24(4), 470–497.
- Wolfram Research. (2024). *WolframAlpha: Computational Knowledge Engine*. https://www.wolframalpha.com

4. **Educational Platforms**
   - Khan Academy. *Mathematics Learning Resources*. https://www.khanacademy.org/math
   - Photomath. *Math Learning App*. https://photomath.com